

Introduction to Machine Learning



Instructor

**Biniyam
Aschalew**

- ▶ Bachelor's degree in Computer Science and AI from KAIST
- ▶ Currently pursuing a master's degree at KAIST

Course syllabus: a four-week roadmap

- ▶ **Week 1 — Mathematical foundations**

Linear algebra, calculus, linear regression, and optimization practice.

- ▶ **Week 2 — Machine learning**

Supervised and unsupervised methods: logistic regression, SVM, random forests, KNN, and (k)-means.

- ▶ **Week 3 — Deep learning**

Multilayer perceptrons, convolutional networks, and recurrent networks.

- ▶ **Week 4 — Foundation models**

Transformers and the ideas behind systems such as GPT.

Lecture 1 builds one learning chain

mathematical objects $\longrightarrow$ predictor functions $\longrightarrow$ matrix
maps

mistakes $\longrightarrow$ gradients $\longrightarrow$ parameter updates

linear regression $\longrightarrow$ generalization

Goal: understand how numbers become predictions, and how prediction errors become corrections.

Shape tells us how values are organized

object	notation	machine-learning example
scalar	$a \in \mathbb{R}$	one temperature or one loss value
vector	$x \in \mathbb{R}^d$	the d features of one example
matrix	$X \in \mathbb{R}^{n \times d}$	n examples, each with d features
tensor	$T \in \mathbb{R}^{n_1 \times \dots \times n_k}$	a batch of color images

The number of axes describes how values are organized; the shape gives the length of each axis.

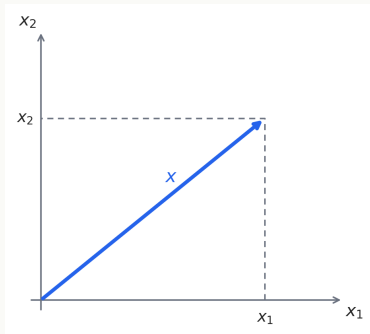
Vectors have coordinates and magnitude

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^d, \quad \|x\|_2 = \sqrt{\sum_{j=1}^d x_j^2}.$$

Distance follows from the same norm:

$$\text{dist}(x, z) = \|x - z\|_2.$$

A norm is meaningful only after feature scales and units are specified.



The dot product is a weighted sum

$$x^{\top} w = \sum_{j=1}^d x_j w_j.$$

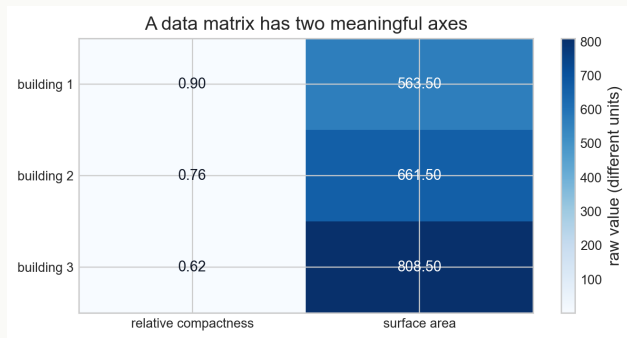
$$x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4$$

Multiply matching coordinates, then add the products. In a linear model, each weight says how strongly its feature contributes to the prediction, holding the other coordinates fixed.

A data matrix has two meaningful axes

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

- ▶ row i : one example;
- ▶ column j : one feature;
- ▶ X_{ij} : feature j of example i .



Lecture companion figure · generated with NumPy + Matplotlib.

Transposition controls which axes meet

If $X \in \mathbb{R}^{n \times d}$, then $X^T \in \mathbb{R}^{d \times n}$.

$$\underbrace{\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \end{bmatrix}}_{2 \text{ examples} \times 3 \text{ features}} \xrightarrow{T} \underbrace{\begin{bmatrix} x_{11} & x_{21} \\ x_{12} & x_{22} \\ x_{13} & x_{23} \end{bmatrix}}_{3 \text{ features} \times 2 \text{ examples}} .$$

The transpose is not decoration: it changes which inner dimensions are aligned.

Predictors map inputs to estimates

A function assigns one output to each valid input:

$$f : \mathcal{X} \rightarrow \mathcal{Y}, \quad x \mapsto \hat{y} = f(x).$$

Input x
compactness, area, orientation,
weather

Predicted output $\hat{y}$
tomorrow's heating load

Machine learning does not invent the idea of a function; it uses data to choose which function should make the predictions.

Supervised data pair features with targets

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R}.$$

example i	features x_i	target y_i
building 1	compactness, area, orientation, ...	heating load
building 2	compactness, area, orientation, ...	heating load

A **feature** is an input coordinate. A **target** is the quantity the training signal asks the model to predict.

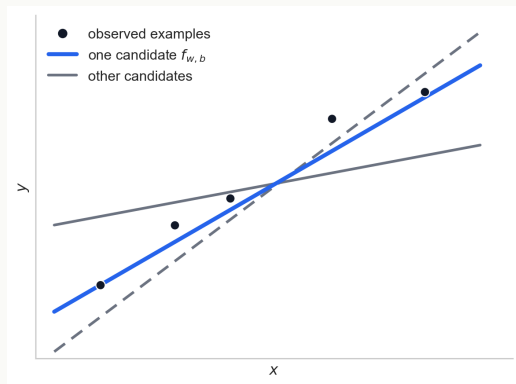
A model is a family of candidate functions

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}, \quad \hat{y}_i = f_\theta(x_i).$$

For affine regression,

$$f_{w,b}(x) = x^\top w + b.$$

The data select w, b from an entire family of possible lines or planes.

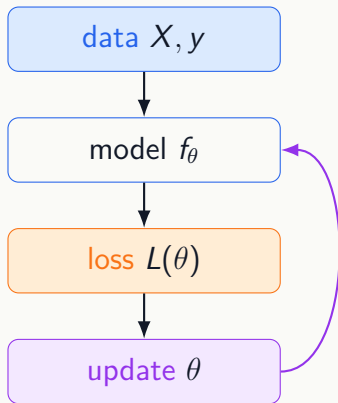


Learning selects parameters from evidence

A model is a family of functions f_θ . Training chooses the member whose predictions fit observed examples best:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

The symbol $\arg \min$ returns a *parameter*, not a loss value.



A matrix represents a linear function

For $A \in \mathbb{R}^{m \times d}$, define

$$T_A : \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad T_A(x) = Ax.$$

The defining property is

$$T_A(\alpha u + \beta v) = \alpha T_A(u) + \beta T_A(v).$$

Rows determine each output coordinate. Columns show where the input basis directions go.

Multiplying by a matrix is applying a function with a known shape.

A data matrix produces all predictions

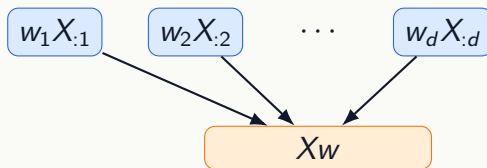
$$Xw = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{bmatrix} w = \begin{bmatrix} x_1^T w \\ x_2^T w \\ \vdots \\ x_n^T w \end{bmatrix} .$$

Row view: one prediction is computed for every example.

The column view of a matrix map

Write the columns of X as $X_{:1}, \dots, X_{:d} \in \mathbb{R}^n$. Then

$$XW = w_1 X_{:1} + w_2 X_{:2} + \dots + w_d X_{:d}.$$



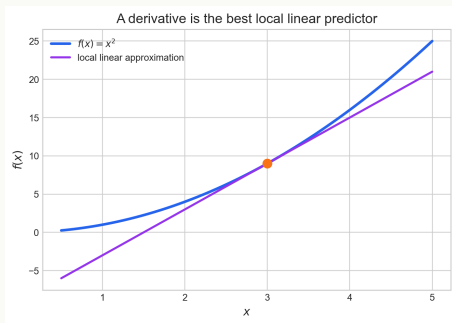
Column view: every prediction vector lies in the span of the feature columns.

Derivatives predict local change

Near x ,

$$f(x + \Delta x) = f(x) + f'(x)\Delta x + o(|\Delta x|).$$

The derivative predicts the first-order change in f , not merely the slope drawn on a graph.



Lecture companion figure · generated with NumPy + Matplotlib.

A gradient predicts first-order change

For $L : \mathbb{R}^p \rightarrow \mathbb{R}$,

$$L(\theta + \Delta\theta) = L(\theta) + \nabla L(\theta)^\top \Delta\theta + o(\|\Delta\theta\|).$$

Equivalently, the differential is

$$dL = \nabla L(\theta)^\top d\theta.$$

The gradient is the vector that converts a small parameter displacement into the predicted first-order change in loss.

The negative gradient points downhill

Substitute $\Delta\theta = -\eta\nabla L(\theta)$ into the local model:

$$\begin{aligned} L(\theta + \Delta\theta) &\approx L(\theta) + \nabla L(\theta)^\top \Delta\theta \\ &= L(\theta) - \eta \|\nabla L(\theta)\|_2^2. \end{aligned}$$

Therefore, for sufficiently small $\eta > 0$,

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

moves downhill unless the gradient is zero.

Linear regression is affine in its inputs

For one example,

$$\hat{y}_i = x_i^\top w + b = \sum_{j=1}^d X_{ij} w_j + b.$$

For the whole batch,

$$\boxed{\hat{y} = Xw + b\mathbf{1}} \qquad \begin{array}{c} (n \times d)(d) + (n) \\ \Downarrow \\ (n) \end{array}$$

Parameters w, b are learned. The feature matrix X and targets y are data.

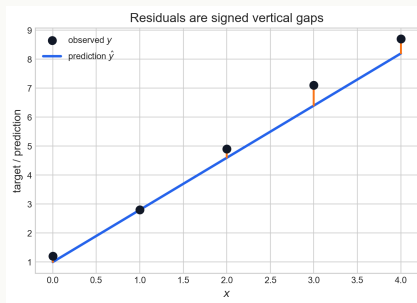
Residuals are signed prediction errors

Our convention is

$$r = \hat{y} - y = Xw + b\mathbf{1} - y.$$

Therefore:

- ▶ $r_i > 0$: model predicts too high;
- ▶ $r_i < 0$: model predicts too low.



Lecture companion figure · generated with NumPy + Matplotlib.

The factor $1/2$ simplifies the derivative

The lecture objective is

$$L(w, b) = \frac{1}{2n} \|Xw + b\mathbf{1} - y\|_2^2 = \frac{1}{2n} r^\top r.$$

Reported mean squared error:

$$\text{MSE} = \frac{1}{n} \|r\|^2 = 2L.$$

Both objectives have the same minimizer because multiplying by a positive constant does not change $\arg \min$.

We will never switch conventions silently.

Empirical risk averages per-example mistakes

Choose a per-example loss $\ell(\hat{y}_i, y_i)$. Then

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

Training asks a narrow question:
which parameters reduce error on
the observed sample?

**Evaluation asks a different
question: does the fitted rule work
on unseen examples?**

Residuals contribute to the gradient

For $\hat{y}_i = wx_i + b$, define $r_i = wx_i + b - y_i$. Then

$$L(w, b) = \frac{1}{2n} \sum_{i=1}^n r_i^2.$$

Chain rule gives

$$\begin{aligned}\frac{\partial L}{\partial w} &= \frac{1}{n} \sum_{i=1}^n r_i \frac{\partial r_i}{\partial w} = \frac{1}{n} \sum_{i=1}^n r_i x_i, \\ \frac{\partial L}{\partial b} &= \frac{1}{n} \sum_{i=1}^n r_i \frac{\partial r_i}{\partial b} = \frac{1}{n} \sum_{i=1}^n r_i.\end{aligned}$$

Residual sign and the factor $1/(2n)$ are now accounted for explicitly.

The batch gradient matches the parameters

With residual vector

$$r = Xw + b\mathbf{1} - y,$$

the derivatives are

$$\boxed{\nabla_w L = \frac{1}{n} X^\top (\hat{y} - y)}, \quad \boxed{\frac{\partial L}{\partial b} = \frac{1}{n} \mathbf{1}^\top (\hat{y} - y)}.$$

- ▶ $X^\top r/n \in \mathbb{R}^d$: one correction for every weight;
- ▶ $\mathbf{1}^\top r/n = \text{mean}(r)$: one correction for the bias.

The gradient has the same shape as the parameter it updates.

Learning repeats: predict, measure, correct

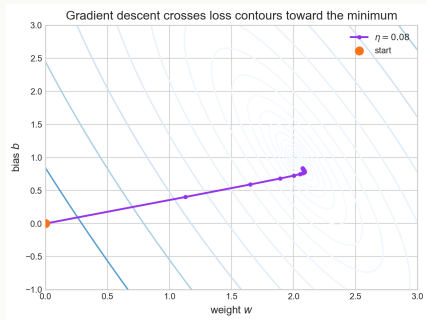
-
1. predict: $\hat{y} = Xw + b\mathbf{1}$
 2. measure mistakes: $r = \hat{y} - y$
 3. summarize them: $L = \|r\|_2^2/(2n)$
 4. compute sensitivity: $g_w = X^\top r/n$, $g_b = \text{mean}(r)$
 5. correct: $w \leftarrow w - \eta g_w$, $b \leftarrow b - \eta g_b$
-

The learning rate $\eta > 0$ controls the size of each correction.

A gradient step crosses loss contours

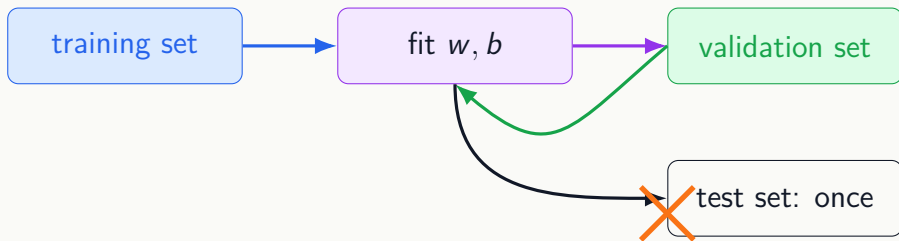
Each contour joins parameters with equal loss.

The gradient is perpendicular to a smooth contour; the negative gradient points across contours toward lower values.



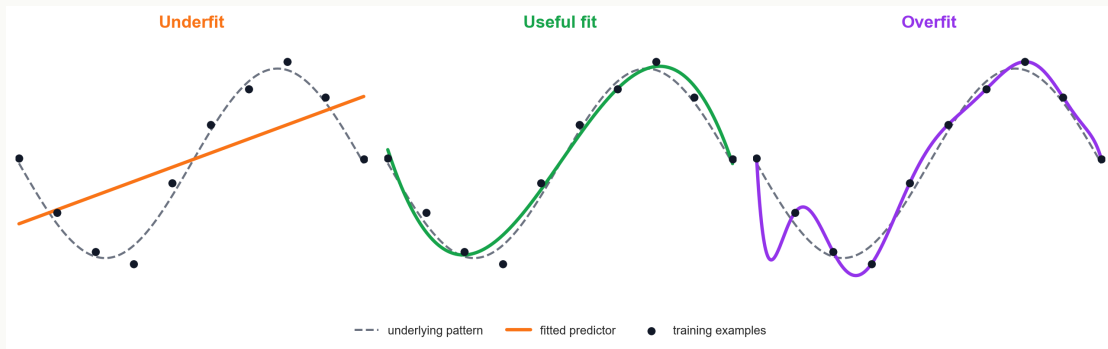
Lecture companion figure · generated with NumPy + Matplotlib.

Evaluation data must not influence fitting



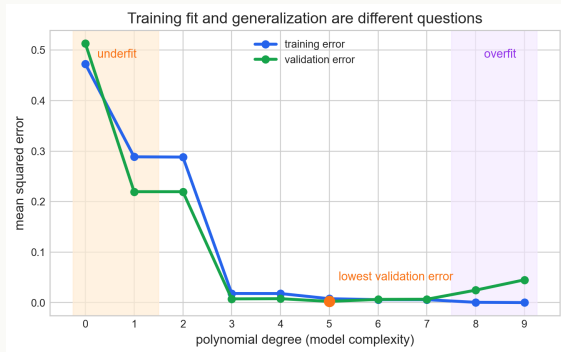
Standardization statistics, feature selection, stopping rules, and learning rates are all part of fitting.

Complexity creates underfitting and overfitting



Validation error reveals overfitting

- ▶ Training error typically falls as flexibility increases.
- ▶ Validation error falls only while the learned pattern transfers.
- ▶ Select complexity using validation data; use the test set once.



Optimization is not generalization

Training question

Did the algorithm reduce $L_{\text{train}}(\theta)$?

Diagnosed with loss curves, gradient norms, and numerical checks.

A perfectly optimized model can still generalize poorly when the model class, sample, or evaluation design is unsuitable.

Scientific question

Does the fitted model predict unseen y accurately?

Estimated with held-out data and a leakage-free evaluation protocol.

The pieces form one learning system

$$x \xrightarrow{f_\theta} \hat{y} \longrightarrow r = \hat{y} - y \longrightarrow L \xrightarrow{\nabla L} \text{parameter update}$$

1. A vector stores one example; a matrix can apply a function to a batch.
2. A gradient predicts how a small parameter change affects the loss.
3. Low training loss is useful only when predictions transfer to unseen data.

Exit check: explain the entire chain without using the word “magic.”